

دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشکده مهندسی مکانیک

پروژه درس شبکه عصبی

دکتر ساداتی

مهندس کشیشیان

مهندس علی ناطقی

شماره دانشجویی : ۴۰۲۱۲۴۲۴

تاریخ تحویل پروژه : ۱۴۰۳/۱۱/۲۳

نیمسال اول ۱۴۰۳-۰۴

موضوع پروژه: تشخیص اعداد دست نویس با استفاده از MNIST

(Modified National Institute of Standards and Technology database)

در این پروژه، یک مدل شبکه عصبی برای تشخیص اعداد دستنویس طراحی و پیاده سازی خواهد شد. هدف اصلی آشنایی با مفاهیم پایه ای مانند لایه های کانولوشن و فرآیند بهینه سازی مدل است. برای این کار از دیتاست MNIST استفاده میشود که شامل ۶۰۰۰۰ تصویر آموزشی و ۱۰۰۰۰ تصویر تست است. هر تصویر دارای ابعاد ۲۸ در ۲۸ پیکسل و به صورت تکرنگ است.

شروع:

در این پروژه از داده های MNIST استفاده کردیم که شامل ۶۰,۰۰۰ تصویر دستنویس از اعداد ۰ تا ۹ برای آموزش و ۱۰,۰۰۰ تصویر برای تست است. هدف از این پروژه طراحی و آموزش یک مدل شبکه عصبی کانولوشنی (CNN) برای تشخیص اعداد دستنویس است. در این گزارش، مراحل کدنویسی و نحوه پیاده سازی مدل تشخیص اعداد ارایه شده است.

وارد کردن کتابخانه ها و داده ها:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist

from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import to_categorical
```

در ابتدا کتابخانه برنامه پایتون را فراخوانی میکنیم که هر کدام جداگانه وارد میشود (بدلیل اینکه از گوگل کولب استفاده شده است نیازی به وارد کردن کتابخانه های فعلی نیست)

tensorflow: این کتابخانه اصلی برای ایجاد و آموزش مدل های شبکه عصبی است. ما از آن برای ساختن و آموزش مدل CNN استفاده می کنیم. مخفف CNN به معنای Convolutional Neural Network است که به فارسی به آن «شبکه عصبی کانولوشنی» می گویند.

numpy: کتابخانه‌ای است برای انجام محاسبات عددی که در اینجا برای پردازش داده‌ها و منظور کردن آرایه‌ها و تنظیم ابعاد استفاده شده است.

matplotlib.pyplot: برای رسم نمودارها و نمایش تصاویری از داده‌ها استفاده می‌شود.

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

keras.datasets: این ماژول برای بارگذاری دیتاست‌ها استفاده می‌شود. در اینجا از دیتاست MNIST که شامل تصاویر دست‌نویس ارقام است، استفاده می‌کنیم. شامل ۶۰,۰۰۰ تصویر آموزشی و ۱۰,۰۰۰ تصویر تست از اعداد ۰ تا ۹ است. تصاویر اندازه ۲۸×۲۸ پیکسل دارند و سیاه‌وسفید هستند.

در اینجا، داده‌های MNIST بارگذاری می‌شوند که شامل دو بخش است: آموزش و تست

X_train: تصاویر آموزش ۲۸ در ۲۸ پیکسل، ۶۰,۰۰۰ تصویر

این متغیر شامل تصاویر آموزشی است. این تصاویر به صورت آرایه‌های ۲۸ در ۲۸ پیکسلی هستند و به صورت تک‌کاناله (تک‌رنگ) ذخیره می‌شوند. یعنی هر تصویر یک ماتریس ۲۸ در ۲۸ است که هر عنصر آن نشان‌دهنده شدت پیکسل در یک نقطه از تصویر است. به‌طور کلی، ابعاد این داده‌ها به شکل زیر است:

y_train: برچسب‌های آموزش (ارقام دست‌نویس از ۰ تا ۹)

این متغیر شامل برچسب‌ها (labels) یا کلاس‌های صحیح برای هر تصویر در **X_train** است. هر برچسب یا هر کلاس نمایانگر عدد دست‌نویس موجود در تصویر است.

X_test: تصاویر آزمایش ۲۸ در ۲۸ پیکسل، ۱۰,۰۰۰ تصویر

این متغیر شامل تصاویر آزمایشی است که در واقع برای ارزیابی عملکرد مدل بعد از آموزش استفاده می‌شوند.

y_test: برچسب‌های آزمایش

این متغیر شامل برچسب‌های آزمایشی است که درست مانند **y_train** هستند، اما برای داده‌های آزمایشی یا همان کلاس‌ها استفاده می‌شود.

نرمالایز کردن و Reshaping :

```
X_train.shape      # برای اینکه شکل تابع مورد آموزش را متوجه بشیم  
X_test.shape  
X_train_plus = np.expand_dims(X_train, axis = -1 ) / ۲۵۵  
X_test_plus = np.expand_dims(X_test, axis = -1 ) / ۲۵۵  
X_train_plus.shape
```

X_train.shape ← تعداد ۶۰۰۰۰ تصویر دست نویس
(60000, 28, 28)

X_test.shape ← تعداد ۱۰۰۰۰ تصویر تست
(10000, 28, 28)

داده‌های تصاویر به صورت ۰ تا ۲۵۵ در پیکسل‌ها هستند. ما این مقادیر را به بازه [۰, ۱] نرمال می‌کنیم. برای نرمال سازی

شبکه‌های CNN نیاز به ورودی چهاربعدی دارند: (تعداد تصاویر، ارتفاع، عرض، تعداد کانال‌ها)

(np.expand_dims(X_train, axis = -1) برای اینکه مدل CNN داده‌ها را به درستی پردازش کند، باید ابعاد داده‌های ورودی (۶۰۰۰۰, ۲۸, ۲۸) به شکل زیر تغییر کند (یعنی ۲۸ در ۲۸ پیکسل و ۱ کانال رنگی که برای تصاویر سیاه و سفید مناسب است برای RGB عدد ۳ در نظر می‌گیریم) چون کانال رنگی نداریم و فقط سیاه و سفید است، این دستور به شکل (۶۰۰۰۰, ۲۸, ۲۸, ۱) تغییر می‌کند.

تبدیل کلاس ها به فرم کدگذاری One-Hot encoding

```
Y_train_plus = to_categorical(y_train, ۱۰)  
Y_test_plus = to_categorical(y_test, ۱۰)
```

تابع to_categorical

تابع فوق برچسب‌های کلاس (ارقام ۰ تا ۹) را به فرم One-Hot encoding تبدیل می‌کند. به عنوان مثال، برچسب ۳ به صورت [۰, ۰, ۰, ۰, ۱, ۰, ۰, ۰, ۰, ۰] تبدیل می‌شود. این عمل باعث می‌شود کلاس ها را به صورت مقادیر ترتیبی اشتباه نگیرد و ایرادات و نقص در یادگیری کاهش یابد. با استفاده از این تابع مدل به درستی درک می‌کند که هر عدد فقط یک کلاس مجزا است و ترتیب خاصی بین آن‌ها وجود ندارد.

```
CNN = Sequential()
```

تابع Sequential

در **Keras**، مدل‌های شبکه عصبی می‌توانند از دو نوع اصلی ساختار استفاده کنند:

۱. مدل‌های Sequential این مدل‌ها به صورت خطی ساخته می‌شوند، یعنی هر لایه به لایه قبلی متصل است.

۲. مدل‌های Functional API برای ساختارهای پیچیده‌تر که لایه‌ها به صورت غیرخطی به هم متصل می‌شوند، از این مدل استفاده می‌کنیم.

چون که مدل‌های ما بصورت خطی و لایه به لایه مدلسازی می‌شود، که امکان این بتوانیم هر لایه به لایه قبل بصورت خطی اتصال یابد از این مدل استفاده شده است.

زیر شاخه های مدل فوق به صورت زیر است که در ادامه بصورت جزئی تر بررسی میگردد.

```
CNN.add(Conv2D(filters = ۱۲, kernel_size = (۳, ۳), activation= 'relu',  
input_shape = (۲۸, ۲۸, ۱)))  
CNN.add(Conv2D(filters = ۱۶, kernel_size = (۳, ۳), activation= 'relu'))  
CNN.add(MaxPooling2D(pool_size=(۲,۲)))  
CNN.add(Flatten())  
CNN.add(Dense(۱۶, activation = 'relu'))  
CNN.add(Dense(۱۰, activation = 'softmax'))
```

در این قسمت مدل شبکه عصبی کانولوشنی به ترتیب ساخته می شود. اجزای مختلف مدل به شرح زیر است:

Conv2D: این لایه برای اعمال فیلترهای کانولوشنی بر روی تصاویر استفاده می شود. در اینجا، اولین لایه Conv2D با ۱۲ فیلتر و اندازه کرنل ۳ در ۳ اعمال می شود. تابع فعال سازی آن ReLU است که به مدل کمک می کند تا ویژگی های مهم تصویر را استخراج کند.

ReLU ساده ترین و سریع ترین تابع فعال سازی است. این تابع تنها یک محاسبه ساده انجام می دهد: اگر ورودی مثبت باشد، همان ورودی را می گیرد و اگر منفی باشد صفر می شود. این کار باعث می شود که محاسبات سریع تر انجام شوند. در مدل های شبکه عصبی، به ویژه مدل هایی که برای دسته بندی مثل MNIST استفاده می شوند، ReLU معمولاً به عنوان تابع فعال سازی برای لایه های پنهان (hidden layers) استفاده می شود. همچنین می توانستیم از Leaky ReLU استفاده کنیم تا مشکل مردگی نوران ها هم رفع شده باشد. در مواردی استفاده از sigmoid و Tanh و softmax نیز قابل توجه می باشد.

Filters: تعداد فیلترهایی است که در این لایه استفاده می شوند. در واقع این تعداد مشخص می کند که این لایه چند ویژگی مختلف را از تصویر استخراج می کند. هر فیلتر ویژگی خاصی از تصویر را شبیه سازی می کند. در این مسئله با بررسی اعداد و نمودار های بدست آمده از اینکه تابع over leaning نشده باشد به دو مورد عددی فیلتر مختلف رسیدیم

```
CNN.add(Conv2D(filters = ۱۲, kernel_size = (۳, ۳), activation= 'relu',  
input_shape = (۲۸, ۲۸, ۱)))  
CNN.add(Conv2D(filters = ۱۶, kernel_size = (۳, ۳), activation= 'relu'))
```

```
CNN.add(Conv2D(filters = ۸, kernel_size = (۳, ۳), activation= 'relu',
input_shape = (۲۸, ۲۸, ۱)))
CNN.add(Conv2D(filters = , kernel_size = (۳, ۳), activation= 'relu'))
```

که هر دو مورد نتایج قابل قبولی را برای ما نشان داده است. که نتایج در ادامه با دو مقدار **loss** و **accuracy** سنجیده شده است.

kernel_size: سایز و ابعادی فیلتری که مدل در هر مرحله ی منفرد از مدلسازی CNN، تابع بررسی میکند را کرنل سایز گوئیم. در اینجا، فیلترها ۳ در ۳ خواهند بود. که به نظر می رسد قادر است ویژگی های اصلی تصویر را با دقت خوبی شبیه سازی کند.

Activation: به این معنی است که برای این لایه از تابع فعال سازی (ReLU (Rectified Linear Unit استفاده می شود که به مدل کمک می کند ویژگی های غیرخطی را یاد بگیرد و خطی سازی کند.

Input shape: ورودی این لایه به اندازه ۲۸ در ۲۸ پیکسل است که ابعاد تصاویر MNIST هستند و چون تصاویر MNIST تک رنگ (grayscale) هستند، تعداد کانال رنگی برابر با ۱ است.

MaxPooling2D: این لایه برای کاهش ابعاد ویژگی های استخراج شده و کاهش پیچیدگی محاسبات استفاده می شود. از اندازه ۲ در ۲ برای تجمع حداکثر مقدار پیکسل ها استفاده می شود این کاهش ابعاد باعث یادگیری بهتر می شود.

این تنظیم به معنی این است که هر ناحیه ۲ در ۲ از تصویر با هم ترکیب می شود و تنها بزرگ ترین مقدار از آن ناحیه انتخاب می شود. این کار باعث کاهش ابعاد تصویر می شود برای مثال یک تصویر ۲۸ در ۲۸ به ۱۴ در ۱۴ تبدیل می شود و در عین حال ویژگی های اصلی حفظ می شود، همچنین از **Overfitting** جلوگیری شود.

Flatten: این لایه خروجی دو بعدی از لایه های کانولوشنی را به یک بردار یک بعدی تبدیل می کند تا بتوان از آن در لایه های Dense استفاده کرد. این لایه ماتریس های ۲ بعدی که توسط لایه های کانولوشنی و pooling ایجاد شده اند را به یک بردار یک بعدی تبدیل می کند. این عمل ضروری است زیرا لایه های Dense بعدی باید ورودی های یک بعدی دریافت کنند.

Dense: این لایه ها برای شبکه های fully connected (کاملاً متصل) استفاده می شوند. در لایه اول از ۱۶ نورون و در لایه آخر از ۱۰ نورون (برای ۱۰ کلاس مختلف) استفاده می شود. در لایه از تابع فعال ساز ReLU در لایه آخر از تابع فعال سازی Softmax برای تبدیل خروجی به احتمال های کلاس های مختلف استفاده می شود. تابع softmax احتمال هر کلاس را محاسبه می کند.

تابع Softmax برای تبدیل مقادیر خام (logits) که توسط شبکه عصبی تولید می‌شود به احتمالات استفاده می‌شود. این احتمال‌ها نشان‌دهنده احتمال تعلق ورودی به هر کلاس است و به گونه‌ای تنظیم می‌شود که مجموع همه احتمال‌ها برابر ۱ باشد.

ورودی تابع Softmax یک بردار از مقادیر خام (logits) است که می‌تواند هر عددی باشد، و این مقادیر بعد از اعمال تابع Softmax به احتمال‌ها تبدیل می‌شوند.

خروجی تابع Softmax یک بردار از مقادیر بین ۰ و ۱ است که مجموع آن‌ها همیشه ۱ خواهد بود.

کامپایل کردن مدل برای آماده سازی برای آموزش است

```
CNN.compile(loss= "categorical_crossentropy", optimizer = "adam" ,metrics  
= [ "accuracy"])
```

Loss: در اینجا از categorical_crossentropy به عنوان تابع خطا استفاده می‌شود، زیرا این مسئله یک مشکل چندکلاسه است.

Optimizer: از الگوریتم بهینه‌سازی Adam استفاده می‌کنیم که یک الگوریتم پیشرفته و موثر برای آموزش مدل‌ها است.

metrics: برای ارزیابی مدل از معیار accuracy (دقت) استفاده می‌شود که درصد پیش‌بینی‌های صحیح مدل را نشان می‌دهد.


```
CNN.summary()
```

```
CNN.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 12)	120
conv2d_1 (Conv2D)	(None, 24, 24, 16)	1,744
max_pooling2d (MaxPooling2D)	(None, 12, 12, 16)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 16)	36,880
dense_1 (Dense)	(None, 10)	170

Total params: 38,914 (152.01 KB)

Trainable params: 38,914 (152.01 KB)

Non-trainable params: 0 (0.00 B)

آموزش مدل

این تابع برای شروع فرآیند آموزش مدل استفاده شده است که در این مرحله، مدل روی داده‌های ورودی آموزش داده می‌شود و وزن‌های مدل به روز می‌شوند

```
cm = CNN.fit(X_train_plus, Y_train_plus, batch_size= 64, epochs = 20, validation_data=(X_test_plus, Y_test_plus))
```

fit: این متد برای آموزش مدل استفاده می‌شود. در اینجا مدل با استفاده از داده‌های آموزش $(X_train_plus, Y_train_plus)$ آموزش داده می‌شود.

batch_size: اندازه هر دسته داده در هر مرحله آموزش است. در اینجا اندازه دسته 64 است که میتوان از ۱۲۸ هم استفاده کرد.

epochs: تعداد دوره‌های آموزش است. در اینجا ما ۲۰ دوره آموزش را تنظیم کرده‌ایم.

validation_data: داده‌های آزمایشی برای ارزیابی مدل پس از هر دوره آموزش استفاده می‌شود.

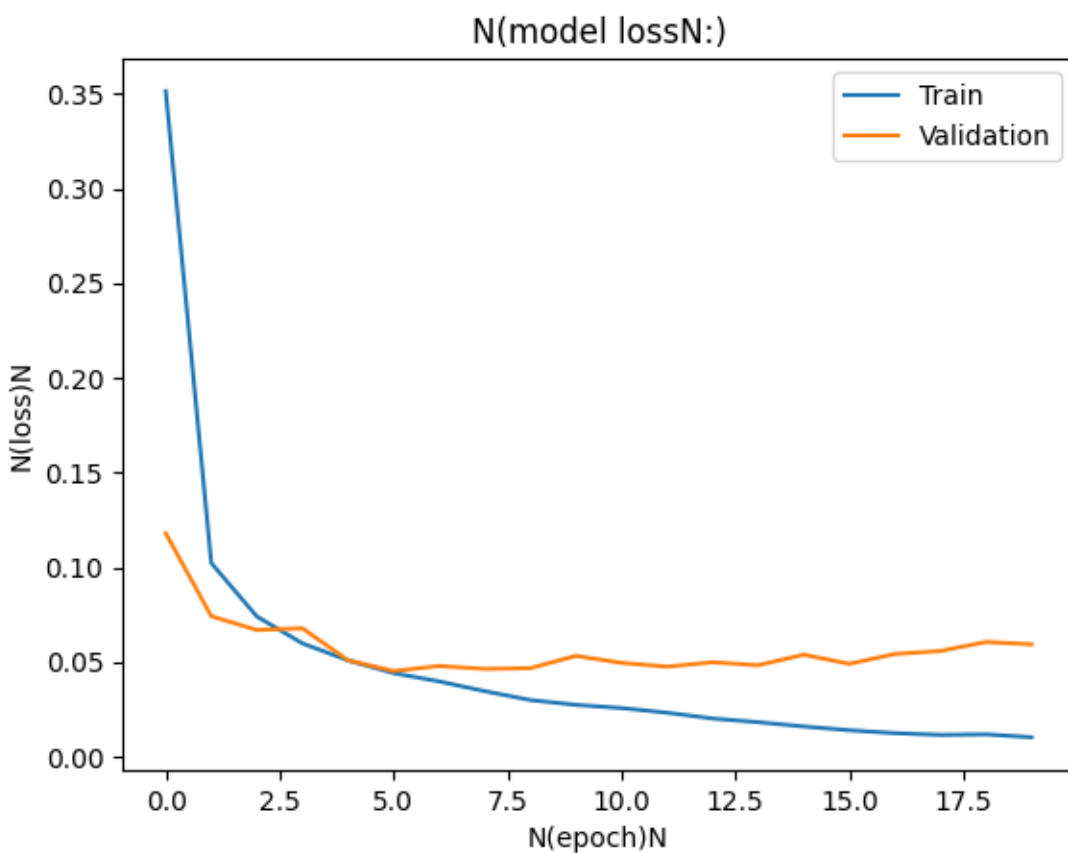
```

Epoch 1/20
469/469 ----- 1s 9ms/step - accuracy: 0.7810 - loss: 0.7242 - val_accuracy: 0.9181
- val_loss: 0.1106
Epoch 2/20
469/469 ----- 1s 1ms/step - accuracy: 0.9110 - loss: 0.1127 - val_accuracy: 0.9711
- val_loss: 0.0719
Epoch 3/20
469/469 ----- 1s 5ms/step - accuracy: 0.9706 - loss: 0.0802 - val_accuracy: 0.9800
- val_loss: 0.0597
Epoch 4/20
469/469 ----- 1s 5ms/step - accuracy: 0.9816 - loss: 0.0627 - val_accuracy: 0.9844
- val_loss: 0.0581
Epoch 5/20
469/469 ----- 1s 5ms/step - accuracy: 0.9848 - loss: 0.0512 - val_accuracy: 0.9837
- val_loss: 0.0526
Epoch 6/20
469/469 ----- 1s 0ms/step - accuracy: 0.9871 - loss: 0.0440 - val_accuracy: 0.9807
- val_loss: 0.0514
Epoch 7/20
469/469 ----- 1s 0ms/step - accuracy: 0.9891 - loss: 0.0340 - val_accuracy: 0.9844
- val_loss: 0.0491
Epoch 8/20
469/469 ----- 1s 5ms/step - accuracy: 0.9899 - loss: 0.0328 - val_accuracy: 0.9830
- val_loss: 0.0521
Epoch 9/20
469/469 ----- 1s 5ms/step - accuracy: 0.9913 - loss: 0.0280 - val_accuracy: 0.9848
- val_loss: 0.0480
Epoch 10/20
469/469 ----- 1s 5ms/step - accuracy: 0.9939 - loss: 0.0227 - val_accuracy: 0.9818
- val_loss: 0.0449
Epoch 11/20
469/469 ----- 1s 0ms/step - accuracy: 0.9920 - loss: 0.0240 - val_accuracy: 0.9816
- val_loss: 0.0484
Epoch 12/20
469/469 ----- 1s 0ms/step - accuracy: 0.9949 - loss: 0.0176 - val_accuracy: 0.9827
- val_loss: 0.0576
Epoch 13/20
469/469 ----- 1s 5ms/step - accuracy: 0.9901 - loss: 0.0168 - val_accuracy: 0.9828
- val_loss: 0.0500
Epoch 14/20
469/469 ----- 1s 5ms/step - accuracy: 0.9906 - loss: 0.0142 - val_accuracy: 0.9807
- val_loss: 0.0540
Epoch 15/20
469/469 ----- 1s 5ms/step - accuracy: 0.9907 - loss: 0.0127 - val_accuracy: 0.9804
- val_loss: 0.0549
Epoch 16/20
469/469 ----- 1s 5ms/step - accuracy: 0.9962 - loss: 0.0124 - val_accuracy: 0.9870
- val_loss: 0.0520
Epoch 17/20
469/469 ----- 1s 1ms/step - accuracy: 0.9970 - loss: 0.0098 - val_accuracy: 0.9817
- val_loss: 0.0521
Epoch 18/20
469/469 ----- 1s 5ms/step - accuracy: 0.9970 - loss: 0.0082 - val_accuracy: 0.9871
- val_loss: 0.0516
Epoch 19/20
469/469 ----- 1s 5ms/step - accuracy: 0.9977 - loss: 0.0074 - val_accuracy: 0.9818
- val_loss: 0.0606
Epoch 20/20
469/469 ----- 1s 0ms/step - accuracy: 0.9979 - loss: 0.0068 - val_accuracy: 0.9872
- val_loss: 0.0506

```

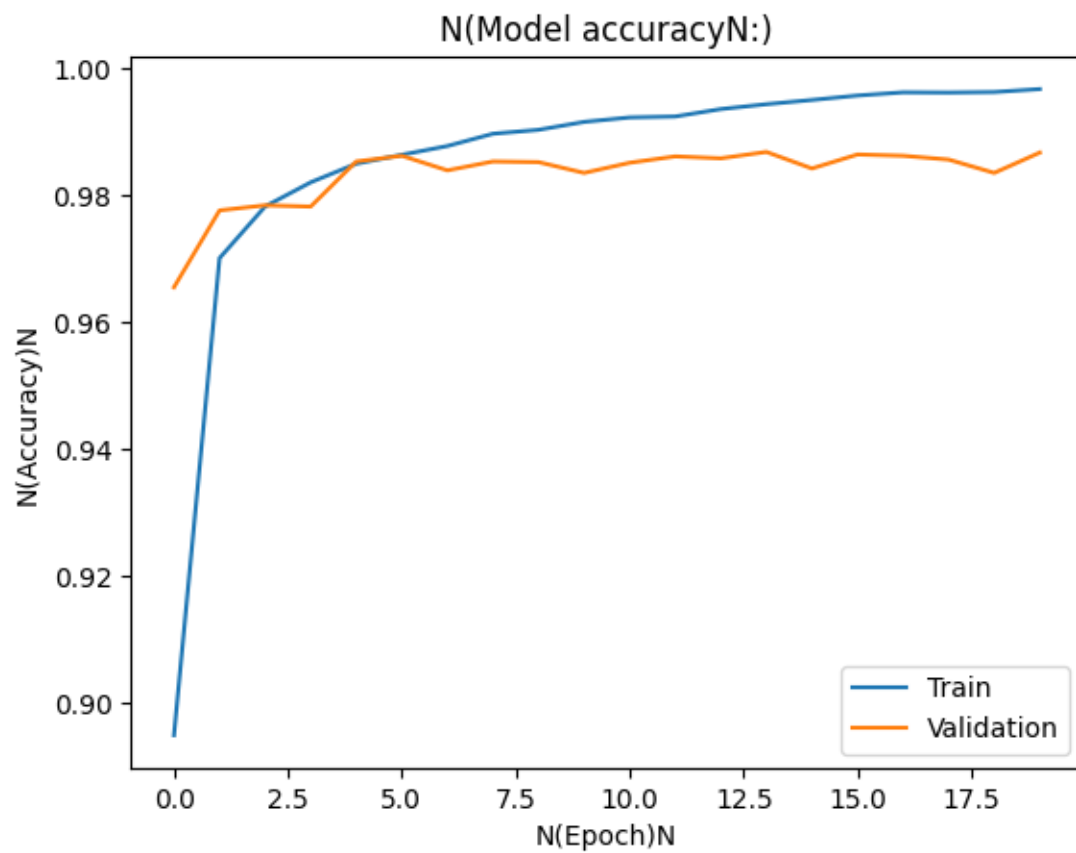
گراف اول (Loss) : این گراف نشان می‌دهد که چقدر خطای مدل در طول آموزش کاهش یافته است.

```
plt.plot(cm.history['loss'])
plt.plot(cm.history['val_loss'])
plt.title('(Nmodel lossN:)' )
plt.xlabel("NepochN")
plt.ylabel("NlossN")
plt.legend(["Train" , "Validation"] , loc = "upper right")
plt.show()
```



گراف دوم (Accuracy): این گراف نشان‌دهنده دقت مدل در هر دوره است و چگونه دقت آموزش و دقت اعتبارسنجی تغییر کرده است.

```
plt.plot(cm.history['accuracy'])
plt.plot(cm.history['val_accuracy'])
plt.title('N(Model accuracyN:)')
plt.ylabel('N(Accuracy)N')
plt.xlabel('N(Epoch)N')
plt.legend(['Train', 'Validation'], loc='lower right')
plt.show()
```



بررسی کلاس ها و پیش بینی هر برچسب مربوط به هر کلاس

```
yp = np.argmax(CNN.predict(X_test_plus), axis=1)
```

برای پیدا کردن کلاس با بیشترین احتمال در هر تصویر از داده‌های تست استفاده می‌شود که `np.argmax` استفاده شده است. در اینجا برای پیدا کردن بزرگترین مقدار در هر ردیف از این تابع استفاده شده است. چون پیش‌بینی‌های مدل به صورت یک آرایه ۲ بعدی است (تعداد تصاویر $\times$ تعداد کلاس‌ها)، هر ردیف (یا هر تصویر) شامل پیش‌بینی‌هایی برای ۱۰ کلاس است.

YP آرایه‌ای است که شامل کلاس‌های پیش‌بینی شده برای هر تصویر در داده‌های تست است یعنی مدل، برای هر تصویر، کدام کلاس (از ۰ تا ۹) را به عنوان پیش‌بینی خود انتخاب کرده است. `axis=1` به این معنی است که شما در هر ردیف (یعنی برای هر تصویر) دنبال بزرگترین مقدار می‌گردید.

ماتریس confusion

```
from sklearn.metrics import confusion_matrix , classification_report  
cm = confusion_matrix(y_test, np.argmax(CNN.predict(X_test_plus), axis=1))
```

```
cm
```

```
print(classification_report(y_test, yp))
```

confusion_matrix : یک ابزار بسیار مهم برای ارزیابی عملکرد مدل است که نحوه عملکرد مدل در پیش‌بینی برچسب‌ها را نشان می‌دهد که در اینجا یک ماتریس ده در ده است ، این ماتریس نشان می‌دهد که مدل چگونه کلاس‌های مختلف را پیش‌بینی کرده است و چه تعداد از پیش‌بینی‌ها درست یا غلط بوده‌اند.

	۰	۱	۲	۳	۴	۵	۶	۷	۸	۹
۰	۹۷۷	۰	۲	۰	۰	۰	۰	۰	۰	۱
۱	۰	۱۱۳۳	۰	۲	۰	۰	۰	۰	۰	۰
۲	۱	۸	۱۰۱۴	۲	۱	۰	۰	۶	۰	۰
۳	۰	۰	۰	۱۰۰۵	۰	۳	۰	۱	۱	۰
۴	۰	۲	۲	۰	۹۷۲	۰	۱	۰	۰	۵
۵	۰	۱	۰	۹	۰	۸۸۰	۲	۰	۰	۰
۶	۳	۴	۱	۰	۱	۲	۹۴۷	۰	۰	۰
۷	۰	۶	۷	۱	۱	۰	۰	۱۰۰۸	۲	۳
۸	۲	۰	۴	۴	۱	۲	۳	۲	۹۵۴	۲
۹	۲	۳	۱	۲	۵	۳	۰	۶	۱	۹۸۶

```
print(classification_report(y_test, yp))
```

`classification_report`: این گزارش شامل معیارهایی مانند دقت (precision)، یادآوری (recall) و F1-score است که عملکرد مدل را به طور کامل تر ارزیابی می کند.

Class	Precision	Recall	F1-Score	Support	Accuracy	Macro Average	Weighted Average
۰	۰٫۹۸	۱	۰٫۹۹	۹۸۰	۰٫۹۹	۰٫۹۹	۰٫۹۹
۱	۱	۱	۱	۱۱۳۵	۰٫۹۹	۰٫۹۹	۰٫۹۹
۲	۰٫۹۹	۰٫۹۸	۰٫۹۹	۱۰۳۲	۰٫۹۹	۰٫۹۹	۰٫۹۹
۳	۰٫۹۹	۰٫۹۹	۰٫۹۹	۱۰۱۰	۰٫۹۹	۰٫۹۹	۰٫۹۹
۴	۰٫۹۸	۰٫۹۹	۰٫۹۹	۹۸۲	۰٫۹۹	۰٫۹۹	۰٫۹۹
۵	۰٫۹۹	۰٫۹۹	۰٫۹۹	۸۹۲	۰٫۹۹	۰٫۹۹	۰٫۹۹
۶	۱	۰٫۹۸	۰٫۹۹	۹۵۸	۰٫۹۹	۰٫۹۹	۰٫۹۹
۷	۰٫۹۸	۰٫۹۹	۰٫۹۹	۱۰۲۸	۰٫۹۹	۰٫۹۹	۰٫۹۹
۸	۰٫۹۹	۰٫۹۹	۰٫۹۹	۹۷۴	۰٫۹۹	۰٫۹۹	۰٫۹۹
۹	۰٫۹۸	۰٫۹۷	۰٫۹۸	۱۰۰۹	۰٫۹۹	۰٫۹۹	۰٫۹۹

دقت خاص: (Precision)

دقت خاص نشان می‌دهد که مدل چقدر در پیش‌بینی صحیح یک کلاس خاص خوب عمل کرده است. برای اکثر کلاس‌ها، دقت خاص مدل نزدیک به ۱ (۱۰۰٪) است که نشان‌دهنده این است که مدل به خوبی توانسته است پیش‌بینی‌های دقیقی انجام دهد. برای مثال، برای کلاس ۱، دقت خاص برابر با ۱۰۰٪ است که نشان می‌دهد از تمام پیش‌بینی‌های کلاس ۱ که مدل انجام داده، تماماً درست بوده‌اند.

یادآوری: (Recall)

یادآوری نشان می‌دهد که مدل چقدر در شناسایی موارد واقعی یک کلاس خاص موفق بوده است. مدل به طور کلی یادآوری بسیار بالایی برای همه کلاس‌ها دارد (نزدیک به ۱۰۰٪)، به این معنی که مدل قادر بوده است بیشتر نمونه‌های واقعی هر کلاس را شناسایی کند. برای مثال، برای کلاس ۱، یادآوری برابر با ۱۰۰٪ است که به این معناست که مدل تمام داده‌های واقعی که متعلق به کلاس ۱ بودند، شناسایی کرده است.

F1 Score :

این معیار متوازن‌تری از دقت خاص و یادآوری است که کمک می‌کند تا عملکرد مدل در هر کلاس بررسی شود. برای تمام کلاس‌ها، بالا است (نزدیک به ۱)، که نشان‌دهنده عملکرد خوب مدل در هر دو جنبه دقت و یادآوری است.

: Support

این مقدار تعداد نمونه‌های واقعی هر کلاس را نشان می‌دهد. برای مثال، کلاس ۱ دارای ۱۱۳۵ نمونه است، در حالی که کلاس ۵ فقط ۸۹۲ نمونه دارد.

: Accuracy

دقت کلی مدل ۰.۹۹ یا ۹۹٪ است. این به این معناست که مدل به طور کلی ۹۹٪ از پیش‌بینی‌های صحیح برای تمام داده‌های تست انجام داده است.

:Macro Average

ماکرو میانگین میانگین دقت خاص، یادآوری برای تمام کلاس‌ها است بدون اینکه تعداد نمونه‌ها در هر کلاس در نظر گرفته شود.

در اینجا، ماکرو میانگین برای تمام معیارها ۰.۹۹ است که نشان می‌دهد مدل به طور کلی در تمام کلاس‌ها عملکرد تقریباً مشابهی دارد.

Weighted Average

میان وزندار میانگین دقت خاص، یادآوری و F1-Score برای تمام کلاس‌ها است که در آن تعداد نمونه‌های هر کلاس (پشتیبانی) به عنوان وزن در نظر گرفته می‌شود.

در اینجا، میانگین وزندار برای تمام معیارها نیز ۰.۹۹ است، که نشان‌دهنده عملکرد پایدار و قوی مدل در تمام کلاس‌ها است، حتی اگر تعداد نمونه‌ها در برخی کلاس‌ها متفاوت باشد.

خسته نباشید

ممنونم از توجهتون

In The Name Of God

Input Library

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist

from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import to_categorical
```

Get Data from Database

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

↗ Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 — 0s 0us/step

000000000000

X_train

↗ [Show hidden output](#)

Dimension of Input

X_train.shape

↗ (60000, 28, 28)

Reshaping for CNN Input

```
X_train_plus = np.expand_dims(X_train, axis = -1 ) / 255
```

```
X_test_plus = np.expand_dims(X_test, axis = -1 ) / 255
```

X_train_plus.shape

↗ (60000, 28, 28, 1)

ONE-HOT

```
Y_train_plus = to_categorical(y_train, 10)
Y_test_plus = to_categorical(y_test, 10)
```

Y_train_plus

↗ array([[0., 0., 0., ..., 0., 0., 0.],
[1., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.],
...,
[0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 1., 0.]])

Middle Layer

```
CNN = Sequential()
```

```
CNN.add(Conv2D(filters = 12, kernel_size = (3, 3), activation= 'relu', input_shape = (28, 28, 1)))
CNN.add(Conv2D(filters = 16, kernel_size = (3, 3), activation= 'relu'))
CNN.add(MaxPooling2D(pool_size=(2,2)))
```

```

CNN.add(Flatten())
CNN.add(Dense(16, activation = 'relu'))
CNN.add(Dense(10, activation = 'softmax'))

CNN.compile(loss= "categorical_crossentropy", optimizer = "adam" ,metrics = ["accuracy"])

```

→ /usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input` super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Summary We dont have any freedzed Item

```
CNN.summary()
```

→ Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 12)	120
conv2d_1 (Conv2D)	(None, 24, 24, 16)	1,744
max_pooling2d (MaxPooling2D)	(None, 12, 12, 16)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 16)	36,880
dense_1 (Dense)	(None, 10)	170

Total params: 38,914 (152.01 KB)

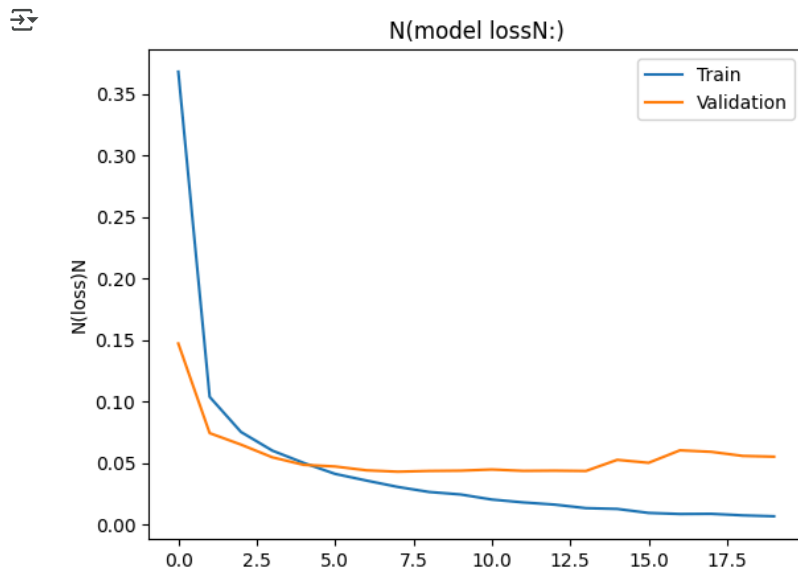
Learn Model

```
cm = CNN.fit(X_train_plus,Y_train_plus, batch_size= 128, epochs = 20, validation_data=(X_test_plus,Y_test_plus),shuffle =True)
```

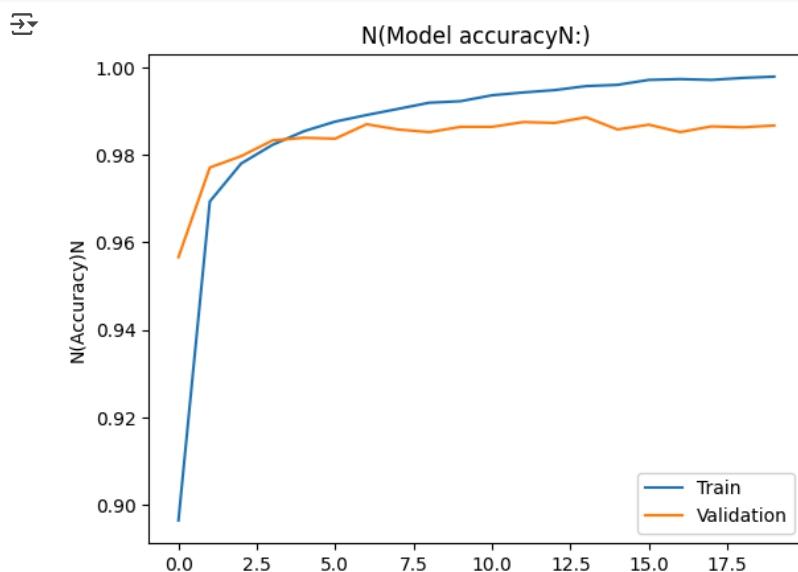
→ Epoch 1/20
469/469 ————— 9s 10ms/step - accuracy: 0.7848 - loss: 0.7493 - val_accuracy: 0.9566 - val_loss: 0.1472
Epoch 2/20
469/469 ————— 2s 4ms/step - accuracy: 0.9665 - loss: 0.1159 - val_accuracy: 0.9771 - val_loss: 0.0743
Epoch 3/20
469/469 ————— 2s 4ms/step - accuracy: 0.9789 - loss: 0.0763 - val_accuracy: 0.9797 - val_loss: 0.0650
Epoch 4/20
469/469 ————— 2s 4ms/step - accuracy: 0.9813 - loss: 0.0620 - val_accuracy: 0.9833 - val_loss: 0.0546
Epoch 5/20
469/469 ————— 2s 4ms/step - accuracy: 0.9859 - loss: 0.0481 - val_accuracy: 0.9839 - val_loss: 0.0485
Epoch 6/20
469/469 ————— 3s 6ms/step - accuracy: 0.9879 - loss: 0.0401 - val_accuracy: 0.9837 - val_loss: 0.0472
Epoch 7/20
469/469 ————— 2s 4ms/step - accuracy: 0.9888 - loss: 0.0366 - val_accuracy: 0.9870 - val_loss: 0.0440
Epoch 8/20
469/469 ————— 2s 4ms/step - accuracy: 0.9918 - loss: 0.0276 - val_accuracy: 0.9858 - val_loss: 0.0429
Epoch 9/20
469/469 ————— 3s 4ms/step - accuracy: 0.9925 - loss: 0.0245 - val_accuracy: 0.9852 - val_loss: 0.0435
Epoch 10/20
469/469 ————— 2s 4ms/step - accuracy: 0.9929 - loss: 0.0236 - val_accuracy: 0.9864 - val_loss: 0.0438
Epoch 11/20
469/469 ————— 3s 5ms/step - accuracy: 0.9939 - loss: 0.0195 - val_accuracy: 0.9864 - val_loss: 0.0447
Epoch 12/20
469/469 ————— 2s 4ms/step - accuracy: 0.9948 - loss: 0.0174 - val_accuracy: 0.9875 - val_loss: 0.0436
Epoch 13/20
469/469 ————— 3s 4ms/step - accuracy: 0.9951 - loss: 0.0147 - val_accuracy: 0.9873 - val_loss: 0.0438
Epoch 14/20
469/469 ————— 3s 4ms/step - accuracy: 0.9962 - loss: 0.0130 - val_accuracy: 0.9886 - val_loss: 0.0435
Epoch 15/20
469/469 ————— 2s 4ms/step - accuracy: 0.9967 - loss: 0.0106 - val_accuracy: 0.9858 - val_loss: 0.0526
Epoch 16/20
469/469 ————— 2s 5ms/step - accuracy: 0.9975 - loss: 0.0086 - val_accuracy: 0.9869 - val_loss: 0.0502
Epoch 17/20
469/469 ————— 2s 5ms/step - accuracy: 0.9979 - loss: 0.0070 - val_accuracy: 0.9852 - val_loss: 0.0603
Epoch 18/20
469/469 ————— 2s 4ms/step - accuracy: 0.9975 - loss: 0.0077 - val_accuracy: 0.9865 - val_loss: 0.0591
Epoch 19/20
469/469 ————— 2s 4ms/step - accuracy: 0.9981 - loss: 0.0055 - val_accuracy: 0.9863 - val_loss: 0.0558
Epoch 20/20
469/469 ————— 2s 4ms/step - accuracy: 0.9983 - loss: 0.0055 - val_accuracy: 0.9867 - val_loss: 0.0551

PLOT

```
plt.plot(cm.history['loss'])
plt.plot(cm.history['val_loss'])
plt.title('N(model lossN:)')
plt.xlabel("N(epoch)N")
plt.ylabel("N(loss)N")
plt.legend(["Train" ,"Validation"] ,loc = "upper right")
plt.show()
```



```
plt.plot(cm.history['accuracy'])
plt.plot(cm.history['val_accuracy'])
plt.title('N(Model accuracyN:)')
plt.ylabel('N(Accuracy)N')
plt.xlabel('N(Epoch)N')
plt.legend(['Train', 'Validation'], loc='lower right')
plt.show()
```



Predict AND MAX OF ROW

```
yp = np.argmax(CNN.predict(X_test_plus), axis=1)
```

313/313 ————— 1s 3ms/step

```
from sklearn.metrics import confusion_matrix ,classification_report
cm = confusion_matrix(y_test, np.argmax(CNN.predict(X_test_plus), axis=1))
```

313/313 0s 1ms/step

Double-click (or enter) to edit

cm

```
array([[ 973,    0,    1,    1,    0,    0,    4,    0,    1,    0],
       [    0, 1125,    2,    2,    0,    0,    1,    1,    4,    0],
       [    1,    1, 1019,    0,    1,    0,    1,    5,    4,    0],
       [    0,    0,    3, 1001,    0,    4,    0,    0,    2,    0],
       [    0,    0,    0,    0,  975,    0,    2,    0,    1,    4],
       [    1,    0,    1,    5,    0, 882,    2,    0,    1,    0],
       [    4,    3,    0,    1,    1,    2,  944,    0,    3,    0],
       [    0,    1,   12,    0,    1,    0,    0, 1010,    1,    3],
       [    5,    0,    1,    4,    1,    0,    1,    1,  959,    2],
       [    2,    0,    0,    3,    7,    6,    0,    4,    8,  979]])
```

export Data to excel

```
import pandas as pd
cmmm = pd.DataFrame(cm)
cmmm.to_csv('cm.csv')
```

print(classification_report(y_test, yp))

	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	1.00	0.99	0.99	1135
2	0.98	0.99	0.98	1032
3	0.98	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.99	0.99	958
7	0.99	0.98	0.99	1028
8	0.97	0.98	0.98	974
9	0.99	0.97	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

GUTTEN **ABEND**